Signals

The addressing layer. How a producer says *what it wants done* without saying *who does it*.

Two fields. Three address modes. One probabilistic step, lazy and cached.

The Primitive

```
{ receiver, data }
```

Unchanged from day one. The whole spec below is about what strings are legal in `receiver` and how the substrate resolves them.

Three Address Modes

```
┌──────────────────┬──────────────────────┬──────────────────────────────────
─┐
│ Mode             │ Example              │ Meaning
│
├──────────────────┼──────────────────────┼──────────────────────────────────
─┤
│ Direct           │ alice                │ Specific unit, default skill
│
│ Direct + skill   │ alice:review         │ Specific unit, named skill
│
│ World + skill    │ world:review         │ Substrate picks the unit
│
│ World + tags     │ world:review+urgent  │ Substrate picks, multi-tag
│
│ Bare world       │ world                │ Strongest highway from
sender│
└──────────────────┴──────────────────────┴──────────────────────────────────
─┘
```

Direct is for **committed relationships** — SOPs, A2A channels, signed envelopes, pipelines that have been promoted to procedure. The producer knows who and why.

`world:` is for **discovery** — "I need this kind of work done; substrate, you choose." The producer names the *kind* of work via skill and tags. The substrate names the *who* via pheromone.

Bare `world` is the rare escape — "I don't know what kind of work this is."
The substrate falls back to the strongest outgoing highway from the sender.
No LLM in the hot path. Ever.

Resolution Flow

```
signal arrives
      │
      ▼
┌──────────────────────────────────┐
│ receiver starts with "world"?    │
└──────────────────────────────────┘
      │                    │
     no                   yes
      │                    │
      ▼                    ▼
┌──────────┐     ┌──────────────────────┐
│ direct:  │     │ parse skill + tags   │
│ deliver  │     └──────────────────────┘
│ as today │                │
└──────────┘                ▼
               ┌──────────────────────┐
               │ candidates with skill? │
               └──────────────────────┘
                     │            │
                    yes           no
                     │            │
                     ▼            ▼
             ┌──────────────┐  ┌──────────────────┐
             │ select()     │  │ enqueue +        │
             │ weighted by  │  │ classify lazily  │
             │ pheromone    │  │ → seed → drain   │
             └──────────────┘  └──────────────────┘
                     │            │
                     └────┬───────┘
                          ▼
                   ┌──────────────┐
                   │ deliver      │
                   │ mark()       │
                   │ on success   │
                   └──────────────┘
```

The direct path is unchanged. The `world:` path is one new branch in `signal()`.
The cold-miss path uses the queue you already have.

The Grammar

```
receiver  := unit | world-addr
unit      := <unit-id> [":" <skill>]
world-addr := "world" [":" <tag-expr>]
tag-expr  := <tag> ("+" <tag>)*
```

Examples:

```
{ receiver: "alice" }                      // direct, default skill
{ receiver: "alice:review" }               // direct, named skill
{ receiver: "world:review" }               // substrate, single tag
{ receiver: "world:review+security" }      // substrate, intersection
{ receiver: "world:review+urgent+P0" }     // substrate, three tags max
{ receiver: "world" }                      // bare — strongest highway
```

Soft norm: keep tag expressions to **three tags or fewer**. More than three is usually a sign you're trying to name a specific agent — use direct addressing instead.

Resolution Rules

`world:skill` with candidates

```
const candidates = unitsWithSkill(skill)
const chosen = select(candidates)   // pheromone-weighted
signal({ ...sig, receiver: chosen.id }, from)
```

`select()` is the existing function. Weight is `strength - resistance` on the edge `from → candidate`, with a small exploration probability. The substrate is already doing this — `world:` just makes it addressable.

`world:tag+tag` (multi-tag intersection)

```
const candidates = units.filter(u => tags.every(t => u.hasTag(t)))
if (candidates.length === 0) {
  // Degrade gracefully — drop the most specific tag and retry.
  return resolve(`world:${tags.slice(0, -1).join("+")}`, from)
}
const chosen = select(candidates)
```

Fail open, not closed. An over-specified tag set should *loosen*, not dissolve.

`world:skill` with zero candidates (cold miss)

```
enqueue(sig)              // queue the original signal
classify(sig).then(seeds => {
  for (const { unit, weight } of seeds) {
    mark([from, unit], weight)   // weak seed, ~0.3
  }
  drain()                  // resignal from the queue
})
```

`classify()` is the only LLM call in the spec. It runs **once per cold miss**, **off the hot path**, and writes its output as pheromone — which means the *next* signal of the same shape routes for free. The pheromone field is the cache.

Bare `world`

```
const chosen = follow(from)   // strongest outgoing edge, any kind
chosen ? signal({ ...sig, receiver: chosen }, from) : dissolve(sig)
```

No classification. No LLM. If the sender has no outgoing highway, the signal dissolves. Bare `world` is best-effort and producers should know it.

For `ask()` — the synchronous contract — bare `world` **must** use this path. Never block `ask` on classification.

Why No Cron

The substrate already runs L1–L7 on tick. Adding cron would mean adding external infrastructure (schedules, locks, idempotency) for something the tick handles.

But we don't even need a tick-scheduled primer. Eager classification spends its budget on tag combinations nobody asks about, while real cold misses wait for the next pass. **Lazy beats eager:** classify on first miss, cache as pheromone, never classify the same shape twice. The signal is its own trigger; the pheromone field is its own cache.

```
Eager primer (rejected):              Lazy on miss (chosen):

  cron every hour                       signal arrives
    walk all skill pairs                  candidates? → select
    LLM-classify each                     no candidates? → enqueue
    write seed weights                      classify once
                                            seed pheromone
```

```
  most never get used                          drain
  all cost up front                       pay only for what's asked
```

No new loop. No cron. No infrastructure. One branch, one queue handler, one cached LLM call per genuinely novel routing question.

The Deterministic Sandwich, Applied to Routing

```
PRE:   receiver = direct?         → deliver, no LLM (existing path)
PRE:   receiver = world:tag?      → candidates? → select, no LLM
PRE:   no candidates?             → enqueue (still no LLM in hot path)
LLM:   classify queued signal     → off-path, writes pheromone
POST:  drain → deliver → mark()   → next time routes for free
```

The LLM is the gardener, not the guard. It only `mark()` s; it never `warn()` s. Resistance is earned by real failure, not predicted by a model. Strength is a hypothesis the substrate can test. Resistance is a verdict the substrate has earned. Keep the asymmetry strict.

When to Use Each Mode

Situation	Use	Why
SOP, audited pipeline	`alice:review`	Variance is forbidden
A2A channel, signed envelope	`alice`	Contract is fixed
Internal pipeline (A always → B)	`alice:next`	Cheap, no resolution cost
"I need code reviewed"	`world:review`	Discovery, pheromone learns
"I need urgent security review"	`world:review+security`	Multi-tag narrows the field
New signal, unknown work type	`world` (rare)	Best-effort highway fallback

The rule of thumb: **direct for commitment, `world:` for discovery, bare `world` almost never.**

Lifecycle of a Relationship

```
discovery                proven                        committed
─────────                ──────                        ─────────
world:review      ──►    world:review                  alice:review
                         (always lands on alice)
LLM classifies           pheromone routes              producer hardcodes
seed pheromone           for free                      SOP / contract
```

A new signal type starts as `world:tag`. The substrate discovers a carrier. Pheromone accumulates. The same emit keeps landing on the same agent — the *address* is still abstract, but the *route* is fixed. Once the relationship is formal enough (compliance, contract, channel), the producer switches to a direct receiver and the substrate gets out of the way.

That's not three patterns. It's one pattern at three levels of commitment. The substrate is the labor market. Direct receivers are the employment contract. Both have to exist.

What Producers Emit

```
// GOOD — discovery, abstract address, substrate decides
emit({ receiver: "world:review", data: pr })

// GOOD — discovery with hint via tag
emit({ receiver: "world:review+security", data: pr })

// GOOD — committed relationship
emit({ receiver: "alice:review", data: pr })

// AVOID — bare world unless you really mean it
emit({ receiver: "world", data: pr })

// AVOID — over-specified tag set (just name the unit)
emit({ receiver: "world:review+security+spanish+P0+nightowl", data: pr })
```

Soft rule for producers: **always include a tag** unless you have a specific reason not to. The norm is `world:something`, not bare `world`.

What Doesn't Change

- `signal()` signature
- `ask()` semantics — still `{ result | timeout | dissolved }`
- `mark()` / `warn()` — still earned by outcomes

- `select()` — still pheromone-weighted
- `fade()` — still asymmetric, still resistance forgives 2x faster
- `then()` continuations — still valid for known chains
- The seven loops L1–L7 — unchanged
- Direct addressing — fully preserved as escape hatch and SOP path

This is purely additive. Every existing signal in the codebase continues to work. `world:` is a new prefix, not a new architecture.

Implementation Sketch

```
// src/engine/world.ts — one new branch in signal()
signal(sig: Signal, from = 'entry') {
  if (sig.receiver === 'world' || sig.receiver.startsWith('world:')) {
    return this.resolveWorld(sig, from)
  }
  // existing direct path unchanged
  const [unitId, skill] = sig.receiver.split(':')
  const target = this.units.get(unitId)
  target?.receive(skill ?? 'default', sig.data, from)
  this.mark([from, unitId])
}

resolveWorld(sig: Signal, from: string) {
  const expr = sig.receiver.slice(6)             // strip "world:" or ""
  if (!expr) {
    const next = this.follow(from)
    return next && this.signal({ ...sig, receiver: next }, from)
  }
  const tags = expr.split('+')
  let candidates = this.unitsWithAllTags(tags)
  while (candidates.length === 0 && tags.length > 1) {
    tags.pop()                                   // loosen
    candidates = this.unitsWithAllTags(tags)
  }
  if (candidates.length === 0) {
    this.enqueue(sig)
    return this.classifyLater(sig, from)         // off-path
  }
  const chosen = this.select(candidates, from)
  return chosen && this.signal({ ...sig, receiver: chosen.id }, from)
}
```

That's the entire change. Everything else — `classifyLater`, the LLM call, the seed marks — reuses primitives that already exist (`enqueue`, `mark`, `drain`).

See Also

- DSL — `signal`, `emit`, `mark`, `warn`, `select`, `follow`
- dictionary — every concept, every name
- routing — how `select()` weighs strength minus resistance
- metaphors — same idea in ant, brain, team, mail, water skins

---

Direct for commitment. `world:` for discovery. Pheromone for everything.